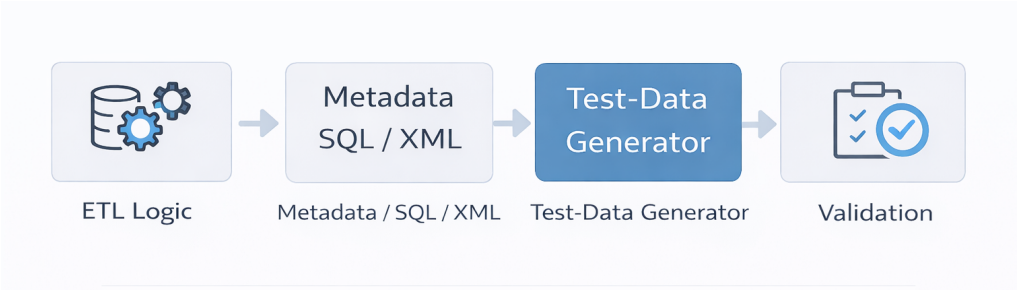
Metadata-Driven Test Data Generation for ETL Modernization

From Manual Test Preparation to Reusable, Metadata-Based Validation



A practical approach to reducing manual ETL test design effort through metadata-driven generation and validation

Modern Data Platforms under Constraint
A HeptaFlow Perspective from Pavel Saratchev
April 3, 2026

Contents

1 Executive Summary 2

2 The Problem: Manual ETL Test Preparation Does Not Scale 2

3 Solution Overview 4

4 Architecture of the Test Data Generator 4

5 Concrete Application Example 6

6 Business Benefits 7

7 Conclusion and Outlook 7

1 Executive Summary

Enterprise ETL modernization is not only a code-conversion challenge. It is also a testing challenge. In many organizations, test data preparation for ETL processes is still largely manual: engineers read mappings and SQL logic, identify relevant scenarios, create source rows by hand, define expected target states manually, and then rework both test data and expected results whenever the ETL process changes.

This approach is slow, expensive, error-prone, and difficult to scale. It creates a repeated validation burden for every migration and every subsequent change to an ETL process.

This paper presents a **metadata-driven test data generator** for ETL modernization. The solution derives test scenarios systematically from metadata, SQL logic, and transformation context instead of relying on handcrafted test cases. It helps ETL engineers generate structured source test data, validate expected target behavior more consistently, and reduce manual rework when processes evolve.

The core business value is straightforward:

- **Lower testing effort** because relevant scenarios are derived systematically instead of being designed from scratch.
- **Faster ETL modernization** because test preparation no longer becomes a separate mini-project for each migration.
- **Less maintenance overhead** because changes in ETL processes no longer require fully manual redesign of test data.
- **Better quality and consistency** because validation is based on explicit metadata and transformation logic rather than individual interpretation.
- **Higher trust in migrated ETL** because engineers can verify important semantics more systematically.

The paper focuses on the current, practical solution: a test data generator built from parsing, normalization, scenario derivation, and validation components. It also outlines how this work can later evolve into a broader verification engine.

Our approach shifts ETL testing from manual, case-by-case design to systematic derivation based on ETL process semantics. This is a critical step toward faster, more reliable, and more maintainable ETL modernization at scale.

2 The Problem: Manual ETL Test Preparation Does Not Scale

In legacy ETL environments, as well as in modern cloud-based environments, testing is often one of the most manual and least automatized parts of the delivery process. Engineers typ-

ically inspect ETL logic, source mappings, filters, joins, key definitions, and target behavior, then manually prepare test cases to cover selected scenarios.

That traditional approach creates several recurring problems:

- Each migration repeats the same validation effort from scratch.
- Each change to an ETL process often requires manual adjustment of the test data and expected results.
- Important lifecycle scenarios are easy to overlook.
- Coverage depends heavily on the individual engineer's experience.
- Test preparation becomes a delivery bottleneck.
- Validation logic is often difficult to reuse across projects.

The contrast between traditional manual test data preparation and a metadata-driven approach is illustrated in Figure 1. It shows how repetitive manual effort can be replaced by a more systematic and significantly faster test-design workflow.

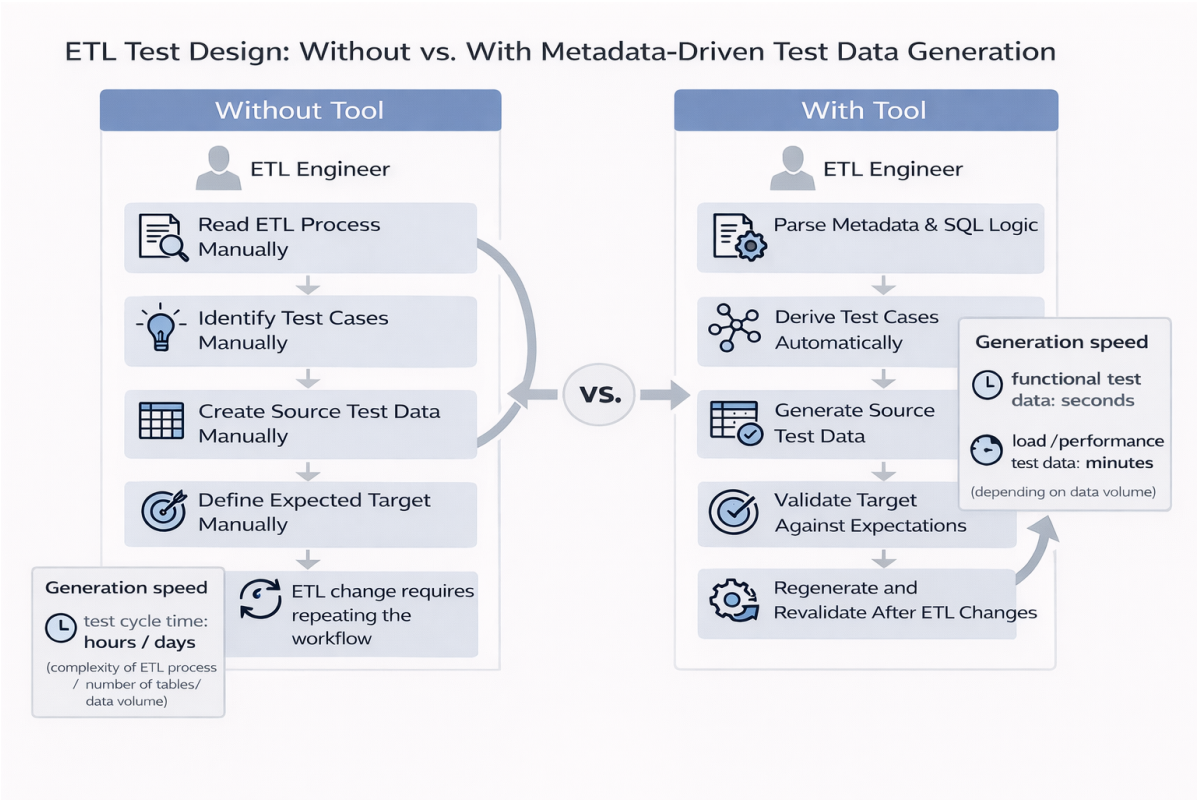


Figure 1: ETL test design without and with metadata-driven test data generation.

This is particularly problematic in enterprise contexts where ETL processes are complex, historized, or highly dependent on key semantics and temporal behavior. A team may successfully convert legacy logic into Spark or another modern execution model, but still struggle to prove that the migrated process behaves correctly under realistic business scenarios.

The result is a familiar pattern: modernization gains speed in implementation, but **testing** remains **slow, fragile, and expensive.**

3 Solution Overview

The metadata-driven test data generator addresses this problem by treating metadata and transformation logic of the ETL processes as the foundation for test preparation.

Instead of asking an ETL engineer to manually invent and maintain test data for each process, the tool leverages structured metadata and transformation logic as the foundation for its preparation. In practice, this includes:

- XML-based process descriptions, defined before implementation or exported from the ETL platform
- Process metadata definitions: process parameters, key definitions, technical attributes, etc.
- SQL transformation logic: filters, joins, aggregations, derivations, etc.
- Dependency structures between source, intermediate, and target objects
- Source-to-target mappings, including business keys and surrogate key semantics

The generator is not just a random data creator. It derives test scenarios directly from transformation semantics and produces **technically meaningful test data** that reflect how the ETL process is supposed to behave. The resulting test data can be exported in multiple formats, including terminal output, JSON, CSV, Parquet, and SQL, making it usable across local analysis, automated validation workflows, and downstream execution environments.

In practical terms, the approach consists of four steps:

1. **Parse the process definition** from metadata and transformation sources.
2. **Normalize and resolve dependencies** so the logic can be interpreted consistently.
3. **Generate structured source-side test data** based on process semantics.
4. **Validate resulting outputs** against expected behavior.

This shifts test preparation from manual interpretation to systematic derivation.

4 Architecture of the Test Data Generator

The architecture of the test data generator is organized as a small set of focused modules that transform process metadata and transformation logic into structured test scenarios, generated test data, and validation outputs. Figure 2 summarizes this flow from source definitions

through parsing, normalization, generation, and export. Each module has a clear role in reducing ambiguity and making the process more systematic.

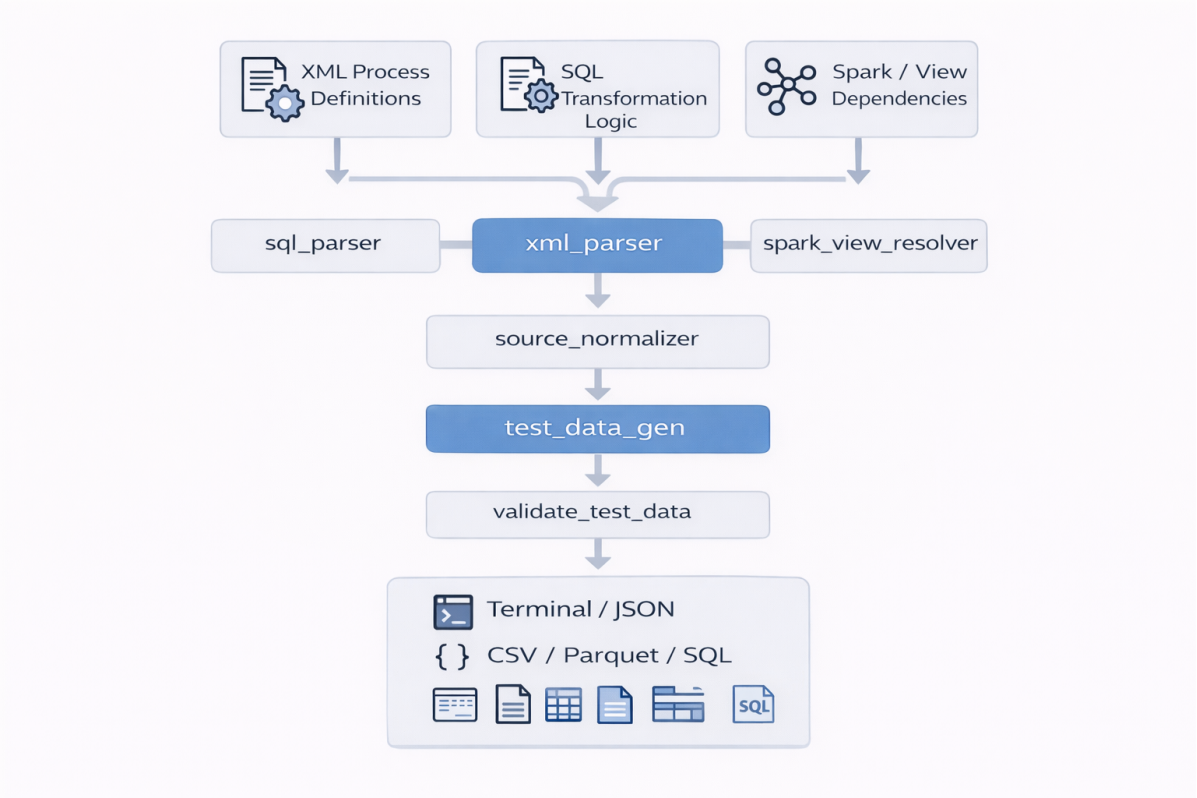


Figure 2: Architecture of the metadata-driven test data generator: from ETL logic and metadata sources to generated test data and validation outputs.

Each module has a clearly defined role in reducing ambiguity and structuring the transformation logic.

- `xml_parser` — Reads structured ETL metadata from XML and extracts the logical process definition, including tables, columns, mappings, and process-specific configuration.
- `sql_parser` — Parses SQL logic so that transformation rules, filters, join relationships, derived expressions, and column-level behavior can be interpreted programmatically.
- `spark_view_resolver` — Resolves view dependencies and traces transformations back to base-level structures.
- `source_normalizer` — Transforms parsed metadata and SQL findings into a canonical DAG-structure that the downstream generation logic can consume consistently.
- `test_data_gen` — Generates source-side test records based on the process semantics.
- `validate_test_data` — Checks whether the generated and processed results match the expected behavior derived from the metadata and transformation context.

Architectural principle

The architectural principle is simple: each module reduces ambiguity.

Raw metadata, SQL, and view logic are often distributed, implicit, and difficult to interpret directly. The architecture turns them into a progressively clearer representation until meaningful test cases can be generated and evaluated.

This modular structure not only reduces complexity but also enables extensibility, allowing the approach to adapt to different metadata representations and processing environments. XML is the current entry point, but the design can later be adapted to other metadata repositories or representations.

5 Concrete Application Example

Consider an ETL pipeline with nine source tables and/or views, some historized and some non-historized. The sources have different temporal characteristics and validity intervals, while the generated test data must still produce consistent time slices across related tables.

In this scenario, the test data generator must support requirements such as:

- generating a defined number of business objects across all source tables,
- producing a controlled number of time intervals for historized sources,
- assigning consistent business keys (BKs) and surrogate keys (SKs) across related records,
- creating test data for both an initial load and a delta load,
- and ensuring that the generated records produce meaningful and predictable results for a complex multi-table join.

In a manual setup, preparing such a scenario is difficult and time-consuming. The engineer must coordinate multiple views and translate them into the underlying tables, preserve key consistency, align historized intervals, and ensure that the resulting records actually activate the intended join paths and target behavior. Even small process changes may require large parts of the dataset to be redesigned manually. The challenge is not only to generate rows, but to produce semantically consistent source states. BKs and SKs must remain aligned across related tables, historized records must follow controlled interval patterns, and the dataset must trigger the intended join behavior in both initial-load and delta scenarios.

With the metadata-driven test data generator, these requirements can be derived systematically from process metadata and transformation logic. The tool can generate a controlled number of business objects and intervals, maintain BK/SK consistency across source structures, create separate datasets for initial-load and delta scenarios, and produce records that are designed to exercise the expected join outcomes.

This makes it possible to test not only isolated rows, but realistic ETL behavior across temporal slices, key relationships, and complex source dependencies.

6 Business Benefits

The value of the test data generator is not only technical. It directly improves delivery speed, quality, and maintainability of ETL modernization efforts.

- **Reduced effort across projects** — Teams do not need to reinvent large parts of the test design for every new ETL implementation or migration. Test scenarios can be derived systematically from existing metadata and transformation logic.
- **Lower cost of change** — When ETL processes evolve, test preparation can be regenerated from updated metadata instead of being manually redesigned from scratch.
- **Better consistency across teams** — Validation becomes less dependent on individual interpretation and more aligned with a shared, systematic approach.
- **Stronger testing of complex ETL behavior** — The generator improves the ability to cover transformation paths, temporal scenarios, and join conditions that are often overlooked in manual testing.
- **Foundation for further automatization** — Once test preparation is metadata-driven, it becomes easier to extend toward broader verification, regression testing, and process-quality automation.

These benefits demonstrate that the value of the approach goes beyond automation. It establishes a more systematic and reliable foundation for ETL testing across projects and teams.

7 Conclusion and Outlook

ETL modernization is not successful when code is merely converted. It is successful when the migrated logic can also be tested with confidence.

Manual test data preparation does not scale in modern migration programs. It is too slow, too dependent on individual interpretation, and too expensive to maintain when ETL processes evolve.

A **metadata-driven test data generator** provides a more industrialized alternative. By parsing process metadata, SQL logic, and dependency structures, it enables the systematic derivation of test scenarios, generation of structured source test data, and validation of expected behavior.

This paper focuses on the current, practical scope: test data generation for ETL processes. At the same time, the approach points toward a broader direction. If metadata and transformation semantics can drive test data generation, they can also serve as the foundation for validation rules, regression testing, and end-to-end verification.

In this sense, the test data generator can evolve into a more comprehensive verification engine. While this broader vision is important, the immediate value already stands on its own: a more systematic, reusable, and scalable way to prepare and validate ETL test data.

The result is a practical step toward faster modernization, lower maintenance effort, and more trustworthy ETL delivery at scale.